Complete Reverse Engineering Checklist

Kill the guesswork, build reflexes, crack binaries faster

🔧 Environment Setup

VM/Container Setup

- [ ] Isolated Analysis Environment
  - Use disposable VM or Docker container
  - Disconnect from network if malware suspected
  - Snapshot VM state before analysis
  - Install essential tools (see Tools section)

Directory Structure

```
/analysis/
├── samples/          # Original binaries
├── notes/           # Running observations
├── scripts/         # Automation scripts
├── extracted/       # Strings, dumps, artifacts
└── test_programs/   # Your verification code
```

🛠️ Must-Have Tools

Core Analysis Stack

- [ ] GDB + Extensions
  - `pwndbg` OR `GEF` OR `peda` (pick one, master it)
  - Learn: `gdb -q ./binary`, `run`, `bt`, `info registers`
- [ ] Disassembler/Decompiler (pick your poison)
  - Ghidra (free, excellent decompiler)
  - IDA (industry standard, expensive)
  - Binary Ninja (modern UI, good for beginners)
- [ ] Radare2/rizin
  - `r2 -d -AA ./binary` for dynamic analysis
  - Essential commands: `afl`, `pdf`, `db`, `dc`, `dr`
- [ ] Linux Essentials
  - `file`, `strings`, `objdump`, `readelf`
  - `strace`, `ltrace` for syscall tracing
  - `hexdump`, `xxd` for raw byte inspection

Setup Verification

- [ ] Compile hello world: `gcc -o test test.c`
- [ ] Open in each tool to verify installation
- [ ] Test debugger: set breakpoint on `main`, run, hit breakpoint

📋 The First 10 Commands (Every Single Time)

1. Basic Reconnaissance

```
file ./binary                    # Format, arch, stripped?
strings ./binary | head -50     # Obvious constants, hints
strings ./binary | grep -i flag # CTF flag hunting
checksec ./binary               # Security mitigations
```

2. Quick Static Peek

```
objdump -d ./binary | head -100  # Disassembly preview
readelf -h ./binary              # ELF header details
readelf -S ./binary              # Section headers
nm ./binary                      # Symbol table (if not stripped)
```

3. Safe Execution Check

```
# In isolated environment only!
strace ./binary 2>&1 | head -20  # Syscall behavior
ltrace ./binary 2>&1 | head -20  # Library call behavior
```

🎯 Static Analysis Workflow

Phase 1: High-Level Mapping

- [ ] Load in main analysis tool (Ghidra/IDA/Binary Ninja)
- [ ] Auto-analysis - let tool do initial lifting
- [ ] Function enumeration - how many functions? Any interesting names?
- [ ] Entry point identification - find `main` or equivalent
- [ ] String references - what strings are used where?

Phase 2: Control Flow Analysis

- [ ] Trace main execution path - happy path first
- [ ] Identify branches - what conditions cause different behavior?
- [ ] Loop identification - for/while loops often process data
- [ ] Function call mapping - what does each function do?
- [ ] Data structure hunting - arrays, structs that might hold secrets

Phase 3: Hypothesis Formation

- [ ] Comment everything - your future self will thank you
- [ ] Rename functions - `process_input()` not `sub_401000`
- [ ] Rename variables - `user_buffer` not `var_10`
- [ ] Document assumptions - "I think this decrypts the flag"

🏃 Dynamic Analysis Workflow

Breakpoint Strategy

- [ ] Entry breakpoint - `break main` or entry point
- [ ] Loop exit breakpoints - catch final processed data
- [ ] Before print statements - data often ready for output
- [ ] Memory allocation - `malloc`, `calloc` calls
- [ ] Suspicious function entries - anything that might transform data

Essential Debugger Commands

GDB + pwndbg/GEF

```
# Starting
gdb -q ./binary
run [args]

# Breakpoints
break main
break *0x401234
info breakpoints

# Execution control
continue (c)
step (s)              # Step into
next (n)              # Step over
finish                # Step out

# Memory inspection
x/20x $rsp           # 20 hex words from stack pointer
x/s $rdi             # String at RDI
x/20i $rip           # 20 instructions from current

# Registers
info registers
print $rax
set $rax = 0x1337
```

Radare2

```
r2 -d -AA ./binary    # Debug mode with full analysis

# Analysis
afl                    # List functions
pdf @ main            # Disassemble main
axt @ sym.flag        # Cross-references to flag

# Debug
db main               # Breakpoint at main
dc                    # Continue
dr                    # Show registers
px 50 @ rax           # Print 50 bytes at RAX
```

Memory Hunting Checklist

- [ ] Register inspection after loops - data often left in registers
- [ ] Stack dumps at function exits - `x/50x $rsp`
- [ ] Following pointer registers - if RDX points somewhere, check it
- [ ] Global variable areas - `.data`, `.bss` sections
- [ ] Heap allocations - follow malloc returns

🔍 Pattern Recognition Guide

Compiler Fingerprints

- [ ] Function prologues - `push rbp; mov rbp, rsp`
- [ ] Stack cleanup - `leave; ret` or manual cleanup
- [ ] Calling conventions - args in RDI, RSI, RDX, RCX (x86-64)
- [ ] Return values - typically in RAX

Common CTF Patterns

- [ ] Array initialization loops - building flag at runtime
- [ ] XOR decryption - `xor` instructions in loops
- [ ] Character-by-character processing - byte-wise operations
- [ ] Conditional printing - flag output after validation
- [ ] Anti-debugging - `ptrace` calls, timing checks

Loop Identification

```
; Typical for loop pattern:
mov ecx, 0            ; counter = 0
jmp check_condition
loop_body:
  ; do work
  inc ecx             ; counter++
```

```
check_condition:
  cmp ecx, length     ; compare with limit
  jl loop_body        ; jump if less
```

🚨 Where to Drop Breakpoints (And Why)

High-Value Breakpoint Locations

1. Right after loops complete
   - Data processing finished, results available
   - `break *0x401234` (instruction after loop)
2. Before print/output functions
   - `printf`, `puts`, `write` calls
   - Data formatted and ready for display
3. After memory allocation
   - `malloc`, `calloc` returns
   - New data structures initialized
4. Function returns with interesting names
   - `decrypt_flag`, `process_input`, etc.
   - Results computed, ready to inspect
5. Conditional branch targets
   - Success/failure paths
   - Different behavior branches

Breakpoint Verification

- [ ] Set breakpoint, continue execution
- [ ] Examine registers: `info registers`
- [ ] Check memory at register addresses: `x/s $rdi`
- [ ] Dump stack for local variables: `x/20x $rsp`

📝 Documentation & Organization

Running Notes Template

```
# Binary Analysis: [filename]
**Date**: [date]
**Objective**: [what am I trying to find?]

## Initial Reconnaissance
- File type:
- Architecture:
- Stripped:
- Security features:
```

```
## Key Functions
- main: [location] - [purpose]
- sub_401000: [renamed to] - [what it does]

## Interesting Strings
- "Enter password": used at [location]
- "Success": printed from [function]

## Current Hypothesis
[What I think is happening]

## Commands Run
```bash
[Keep a log of useful commands]
```
```

Memory Dumps

[Important memory contents found]

```
### Version Control Your Analysis
```bash
cd /analysis/
git init
git add notes/ scripts/ extracted/
git commit -m "Initial analysis setup"

# After each major discovery
git add -A
git commit -m "Found flag construction loop at 0x401234"
```
```

🧪 Verification Labs (Build Your Reflexes)

Lab 1: Compiler Pattern Recognition

```c
// Create this, compile, analyze
#include <stdio.h>
int main() {
    int a = 5, b = 10, c;
    c = a + b;
    printf("Result: %d\n", c);
    return 0;
}
```

- [ ] Compile: `gcc -o lab1 lab1.c`
- [ ] Disassemble, trace each C line to assembly
- [ ] Identify variable storage (registers vs stack)

Lab 2: Loop Analysis

```c
#include <stdio.h>
int main() {
    char flag[] = "abcdefgh";
    for(int i = 0; i < 8; i++) {
        flag[i] ^= 0x42;
    }
    printf("%s\n", flag);
    return 0;
}
```

- [ ] Compile and analyze loop structure
- [ ] Set breakpoint after loop, inspect flag array
- [ ] Verify XOR transformation

Lab 3: Function Call Tracing

```c
#include <stdio.h>
void secret_func(char* data) {
    // Transform data
    for(int i = 0; data[i]; i++) {
        data[i] += 1;
    }
}
int main() {
    char msg[] = "gdkknvnqkc";
    secret_func(msg);
    printf("%s\n", msg);
    return 0;
}
```

- [ ] Trace function calls with `ltrace`
- [ ] Set breakpoints before/after `secret_func`
- [ ] Compare data before and after transformation

🚦 Decompiler Sanity Checks

When Decompiler Output Looks Suspicious

- [ ] Cross-reference with assembly - does the logic match?
- [ ] Check data types - are pointers/arrays correctly identified?
- [ ] Verify control flow - do the conditions make sense?
- [ ] Test edge cases - what if input is empty/invalid?

Red Flags in Decompiled Code

- Weird casts everywhere
- Overly complex expressions for simple operations
- Missing function parameters
- Variables used before initialization
- Impossible memory access patterns

Verification Process

1. Pick suspicious decompiled function
2. Read corresponding assembly manually
3. Write equivalent C code
4. Compile your C and compare assembly
5. Update your understanding, fix decompiler lies

🎯 CTF-Specific Hunting Guide

Flag Format Patterns

- [ ] Search strings for flag format: `CTF{`, `flag{`, `FLAG`
- [ ] Look for base64/hex patterns in strings
- [ ] Check for XOR keys (common: single byte, repeating pattern)

Common Hiding Spots

- [ ] Global arrays - pre-initialized data
- [ ] Stack buffers - built at runtime
- [ ] Heap allocations - dynamic construction
- [ ] Environment variables - `getenv` calls
- [ ] File contents - `fopen`, `fread` sequences

Quick Flag Extraction

```
# After finding flag location in debugger
(gdb) x/s 0x404000      # If flag at known address
(gdb) x/s $rax          # If register points to flag
(gdb) dump memory flag.txt 0x404000 0x404100  # Dump region
```

🔄 Automation Scripts

String Extraction Script

```
#!/bin/bash
# extract_interesting.sh
echo "=== Strings Analysis ==="
strings "$1" | grep -E "(flag|password|key|secret)" -i
```

```
echo -e "\n=== Hex Patterns ==="
strings "$1" | grep -E "^[0-9a-fA-F]{8,}$"
echo -e "\n=== Base64 Candidates ==="
strings "$1" | grep -E "^[A-Za-z0-9+/]{16,}={0,2}$"
```

Quick Analysis Script

```bash
#!/bin/bash
# quick_recon.sh
BINARY="$1"
echo "File info:"
file "$BINARY"
echo -e "\nSecurity:"
checksec "$BINARY" 2>/dev/null || echo "checksec not available"
echo -e "\nFunctions:"
nm "$BINARY" 2>/dev/null | grep -E " T " | head -10
echo -e "\nInteresting strings:"
strings "$BINARY" | head -20
```

⚡ Speed Tips

Keyboard Shortcuts (Learn These)

- Ghidra: `G` (goto address), `Ctrl+E` (edit function), `L` (label)
- IDA: `N` (rename), `G` (goto), `;` (comment), `X` (xrefs)
- GDB: `Ctrl+R` (reverse search), `Ctrl+L` (clear), Tab (autocomplete)

Workflow Acceleration

- Use history: `history | grep gdb` to find past commands
- Alias common commands: `alias r2d='r2 -d -AA'`
- Keep a terminal with analysis directory always open
- Use tmux/screen for persistent sessions

🎓 Daily Practice Routine

5-Minute Warm-up

1. Write simple C program
2. Compile with `gcc -o test test.c`
3. Open in disassembler
4. Identify main function, trace first 10 instructions
5. Predict what each instruction does, verify

Weekly Challenges

- [ ] Monday: Analyze a basic crackme

- [ ] Wednesday: Reverse a simple encryption
- [ ] Friday: Practice dynamic analysis on unknown binary
- [ ] Sunday: Write up findings, document new patterns learned

🔥 Emergency Procedures

When Completely Stuck

1. Step back - re-read the problem statement
2. Restart fresh - new VM, clean analysis
3. Simplify - focus on one function at a time
4. Ask specific questions - "what does this loop do?" not "how does this work?"
5. Reproduce minimally - write C that matches the confusing assembly

When Tools Break

- Have backup analysis tools ready
- Keep portable versions of essential tools
- Document your tool configurations
- Practice with multiple disassemblers

When You Find the Flag

- [ ] Document exactly how - others need to reproduce
- [ ] Clean up your analysis - remove dead ends from notes
- [ ] Write the walkthrough - future you will thank past you
- [ ] Extract the pattern - what technique actually worked?

✅ Success Metrics

You're getting good when:

- [ ] You can identify main function in under 30 seconds
- [ ] Assembly doesn't look like random gibberish
- [ ] You spot XOR loops immediately
- [ ] Decompiler lies don't fool you anymore
- [ ] You write verification programs instinctively
- [ ] Your notes are more valuable than the original binary
- [ ] Other people ask you how you solved it so fast

Remember: Reverse engineering is pattern recognition + tooling + patience. Master the patterns, automate the boring parts, and be ruthlessly systematic. The binary will crack.